Project Report

Database Management Systems Laboratory

CS39202

Be Positive with Lurking Crabs

B^+ Tree over a Custom Buffer Pool Manager with LRU-K Eviction and Latch Crabbing for Concurrency

Group BSDM

(Baseless Systems Data Management)

Name	**Roll No.**
Shrey Patel	23CS10051
Arnav Singh	23CS30009
Arham Bhansali	23CS30007

Contents

1 Objective

The objective of this project is to model, design, and implement the core storage and indexing mechanisms of a modern Database Management System from scratch in C++. The systems are broken into two primary components working in tandem: a Buffer Pool Manager and a generic B+ Tree index.

Specifically, the project aims to:

1. Implement a **B+ Tree Index** directly overlaying disk architecture using textual file storage for nodes to emulate database paging constraints.
2. Develop an **LRU-K Buffer Pool** policy to intelligently manage I/O caching, overcoming the limitations of standard LRU under sequential scan flooding.
3. Ensure high-throughput multi-threaded operations by implementing **Latch Crabbing (Lock Coupling)**, enabling concurrent reads and writes without risking tree corruption or deadlocks.

2 Methodology

2.1 Standard Libraries Used & Justifications

This system is built purely in C++ relying exclusively on the highly-optimized C++ Standard Template Library (STL). No external indexing or database libraries were utilized.

- `<fstream>` **and** `<iostream>`: Utilized as the fundamental disk-interaction protocol. Our architecture mimics "pages" as physical text files on disk. These libraries read and serialize nodes reliably.
- `<filesystem>`: For automated creation and management of the `Index/` and `Data/` directories, as well as checking file existence and size.
- `<map>`: The core structure of the nodes. Since a B+ tree requires maintaining ordered keys within each node, `std::map<int, string>` perfectly maintains sorted invariants dynamically while ensuring $O(\log n)$ boundary searches within a node.
- `<queue>` **(Priority Queue)**: Used for recycle-file management (`availPointer`). File name IDs range from 1 to 37449. When nodes are deleted, their file ID is recycled. A priority queue maintains these IDs to continually reuse smaller integer names.
- `<shared_mutex>` **and** `<mutex>`: Essential for the concurrency system. `shared_mutex` enables Multiple-Reader Single-Writer (MRSW) locks mapped to independent buffer pool frames to achieve lock coupling.

2.2 B+ Tree Structure Details

The underlying B+ Tree diverges from solely in-memory models by mapping nodes structurally to individual disk files.

2.2.1 Tree Constraints

- **Order (N):** The tree is defined natively with $N = 8$.
- **Depth Constraints:** The tree restricts expansion to a maximum of 6 levels to bound operations to local hardware storage restrictions.
- **Limit Constraints:** Given order 8 and 6 levels, the system accounts for $\sum_{i=0}^{5} 8^i = 37,449$ maximum nodes. Assuming worst-case half-full leaves, the system strictly enforces a maximum capacity of 131,072 SQL entries.

2.2.2 File and Mapping Layout

Files are split logically into two directories:

- `Index/`: Stores non-leaf and leaf B+ Tree nodes. The name is exactly integer-based (e.g., `Index/42.txt`).
- `Data/`: Stores the actual logical database rows, named using the exact record keys.
- `metadata.txt`: Acts as the persistence catalog for the tree's global state, storing the root file pointer, `numRows`, `currLevel`, and the serialized state of the recycled `availPointer` queue.

File Serialization Protocol:
Each node file begins with a boolean flag mapping to `isLeaf`. Subsequently:

- **Leaf Nodes (`isLeaf = 1`):** The file is serialized as:

$$1 \quad k_1\ p_1 \quad k_2\ p_2 \quad \cdots \quad k_n\ p_n$$

  where each k_i is a record key and p_i is its associated `Data/` file pointer.

- **Internal Nodes (`isLeaf = 0`):** The file is serialized as:

$$0 \quad p_0 \quad k_1\ p_1 \quad k_2\ p_2 \quad \cdots \quad k_n\ p_n$$

  where p_0 is the left-most child pointer (stored under key index 0 in the map), and each subsequent k_i, p_i pair denotes a separator key and its right child pointer.

  Standard B$^+$ Tree Ordering Invariant: For a node with pointers and keys $p_0,\ k_1,\ p_1,\ \ldots,\ k_n,\ p_n$, the following must hold for all i:

$$\max(\text{keys in } p_{i-1}) < k_i \leq \min(\text{keys in } p_i)$$

  Implementation Assumption: All record keys satisfy $k_i > 0$. This is required to unambiguously reserve key index 0 in the internal node map as the dedicated slot for the leftmost child pointer p_0, keeping the serialization protocol consistent without a separate pointer field.

2.3 Search Algorithm with Latch Crabbing (Read Crabbing)

First, the data cache is check for the presence of the search key, by first locking the Page Table of the cache. If not found, then the B+ Tree is searched. Concurrency is achieved through cautious "Latches" independent of strict Locks. Readers operate on an optimistic crabbing protocol:

1. **Root Initialization:** The active thread acquires a **Shared (Read) Latch** on the root buffer frame.
2. **Crabbing Downwards:** At the current node, the thread computes the target child pointer using `upper_bound`. It fetches the child from the Buffer Pool, waiting for I/O if necessary.
3. **Shared Transfer:** Crucially, the thread acquires the Shared Latch on the target *child* before releasing the Shared Latch on the *parent*. This overlapping hold period prevents a writer from modifying the parent-child linkage pointer while the reader is actively traversing it.
4. **Leaf Identification:** Upon reaching the target leaf, the thread finalizes the reading of the data-file pointer and subsequently unlocks the final leaf latch. Multiple readers can share these latches continuously.

2.4 Insert Algorithm with Latch Crabbing (Write Crabbing)

Tree additions modify the node arrays and potentially cause node overflow. Latch crabbing here aims to minimize locking the whole tree.

1. **Root Initialization:** Acquire an **Exclusive (Write) Latch** on the root.
2. **Crabbing Downwards:** For each child node entered, immediately acquire an Exclusive Latch.
3. **Safety Constraint Check:** To minimize bottlenecks, the thread evaluates if the child node is "Safe". For an Insert operation, a node is deemed *Safe* if its current length is $< N$. This guarantees a split will not cascade past this node.
4. **Pessimistic Release:** If the node is *Safe*, the thread immediately releases all held latches on ancestor nodes. If unsafe, the thread retains latches on all ancestors to ensure safe cascading splits.
5. **Leaf Split Logic:** The new key is passed into the physical map. If size $> N$, the system calls `split_node`, carving out $(N + 2)/2$ entries to a new allocated file ID fetched from `availPointer`. The median key is cascaded up synchronously to the safely locked parent. Once resolved, remaining latches are released.

At the end, the file is also inserted in the data cache.

2.5 Delete Algorithm with Latch Crabbing

Deletion embodies the highest structural complexity as it addresses redistributions (Borrowing) and collapsing nodes (Merging).

1. **Root Initialization:** The thread begins exactly as insertion by acquiring an **Exclusive Latch** on the root node and crabbing downwards.
2. **Safety Constraint Check:** For a Delete operation, a node is deemed *Safe* if its element count strictly exceeds the minimum property bound ($\lceil N/2 \rceil$ for leaves, $\lfloor N/2 \rfloor - 1$ for internals). If safe, all ancestor Exclusive latches are released.
3. **Leaf Operation:** The requested primary key is erased, and the row file under `Data/` is recycled.
4. **Underflow & Latch Escalation:** Should an underflow occur:
   - **Borrowing (`borrow_node`):** The system latches an adjacent sibling. If the sibling possesses keys above the minimum margin, a key is shifted to the target leaf, and the shared parent pivot key is actively replaced without altering structural node counts.
   - **Merging (`merge_nodes`):** If the sibling lacks keys to spare, the node mappings are pushed into the sibling file. The target file is entirely removed from the file system and its ID pushed dynamically to `availPointer`. The parent safely strips the pivot key mapping down.
5. **Post-resolution:** Once structural integrity satisfies tree invariants entirely up to the lowest retained safe node, active exclusive latches are simultaneously dropped.

If the key was present in the data cache as well, it is deleted from there.

2.6 LRU-K Implementation Details with Locking

To protect the buffer pool against sequential-scan cache pollution (popular in database range scans), we augmented the standard LRU replacement with the LRU-K algorithm.

- **Algorithm State:** Each resident frame tracks an array of its last K access timestamps.
- **Eviction Logic:** When a miss requires eviction, the system evaluates all unpinned frames. It computes the "Backward K-Distance" (the time difference between the current moment and the K-th historic access). The frame with the maximum Backward K-Distance is chosen.
- **Infinity Handling:** Frames referenced $< K$ times hold an infinite distance by default and are strictly evicted first via standard LRU metrics among themselves.
- **Locking Constraints:** A `std::mutex` covers the Page Table mappings mapping 'file_id $\rightarrow$ frame_id', preventing concurrent threads from allocating parallel frames for the same physical page. Simultaneously, buffer pool counters, pin tracking, and dirty-writebacks execute atomically underneath this table lock.

2.7 B+ Tree State Recovery

Ensuring durability across system restarts is managed by the recovery logic implemented within `bptree_state.cpp`. Upon instantiation, the DBMS reads `metadata.txt` to reconstruct the exact global state of the B+ Tree. This process synchronizes the root node pointer, integer bounds (`currLevel` and `numRows`), and reconstructs the `availPointer` priority queue containing recycled file IDs. Because the nodes themselves are persistently stored as independent files on disk, restoring this metadata is sufficient to recover the entire tree topography without recreating the index from scratch.

2.8 Overall Folder Structure and Makefile

The C++ software layout is structured professionally for modular compilation.

```
/project_root
|-- Makefile              # Compilation directives
|-- README.md
|-- metadata.txt          # Globals: root ID, limits, recycled files
   queue
|-- include/              # Core Classes, Structs, Template Definitions
    |-- bptree.hpp
    |-- lru_k.hpp
|-- src/
    |-- insert.cpp        # Crabbing insertion and node splittings
    |-- delete.cpp        # Crabbing deletion, merge, borrow limits
    |-- search.cpp        # Crabbing point search retrieval
    |-- file_ops.cpp      # Disk abstraction and buffer pool hooks
    |-- bptree_state.cpp  # Metadata parsing and tree state recovery
    |-- lru_k.cpp         # Buffer pool and eviction frame logic
    |-- main.cpp          # CLI test runner and benchmarking hook
|-- Index/                # Active B+ Tree Node pages (e.g. 5.txt)
|-- Data/                 # Active SQL Row entry pages
|-- apartments_for_rent_classified_100K.csv
                          # Sample data file
|-- benchmark/
    |-- run_experiment_sim1.sh
    |-- run_experiment_sim2.sh
    |-- run_experiment_sim3.sh
|-- Results/
    |-- experiments_results_sim1.txt
    |-- experiments_results_sim2.txt
    |-- experiments_results_sim3.txt
|-- build/
```

Make Pipeline Details: The `Makefile` builds target execution files using `g++ -std=c++20 -Wall -Wextra -pthread`. The standard sequence targets `src/*.cpp`, binds them statically referencing representations from `include/`, and outputs an environment-agnostic executable, maintaining high cache-hit bounds for local benchmarking.

3 Results and Benchmarks

3.1 Database and Workload Configuration

To evaluate the performance of our custom B+ Tree and LRU-K buffer pool, we designed a series of benchmarking simulations. The underlying database is pre-populated with a set distribution of records to simulate a steady-state environment before measurements begin. The buffer pool is logically split into an **Index Cache** (managing the node pages) and a **Data Cache** (managing the actual row pages). Across all simulations, the Data Cache capacity is held constant at 1,000 frames to isolate the impact of index traversal and caching policies. Concurrent operations are mixed to reflect realistic transactional workloads, allowing us to evaluate the efficiency of our latch crabbing and eviction constraints.

3.2 Simulation 1: Analyzing the Impact of K in LRU-K

This simulation evaluates how varying the history parameter K (from 0 to 5) in the LRU-K eviction policy affects both operational latency and cache hit ratios.

Simulation Parameters:

- **Index Cache Frames:** 100 frames
- **Data Cache Frames:** 1,000 frames
- **Total Operations Performed:** 100000
- **Workload Distribution:** 40% Search, 30% Insert, 30% Delete

Visualizations:

1. **Graph 1: Average Latency vs.** K: A combined line graph plotting the average time taken per Insert, Delete, and Search operation against K.

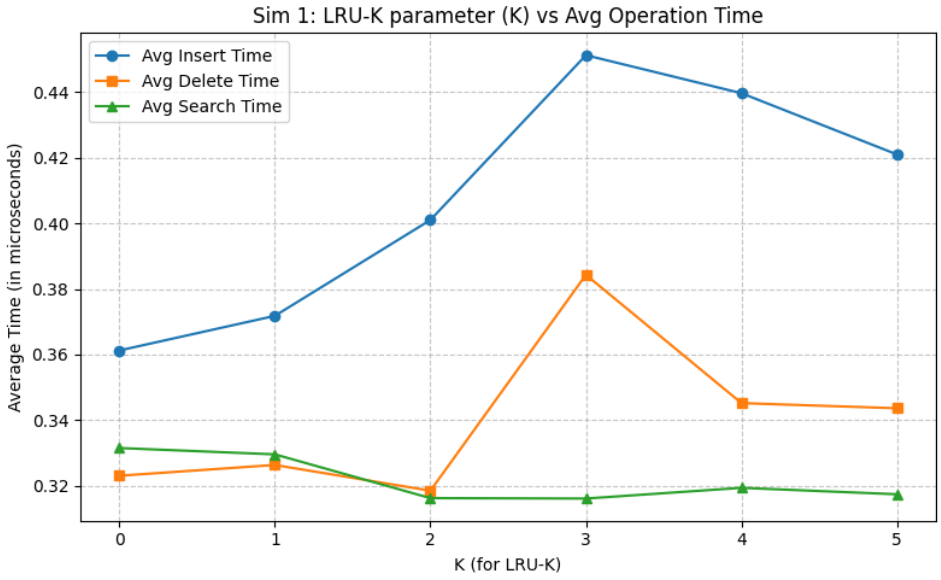

Figure 1: Simulation 1: Avg operation time v/s K

2. **Graph 2: Hit Ratios vs.** K: A combined line graph plotting the total hit ratio for both the Index Cache and Data Cache against K.

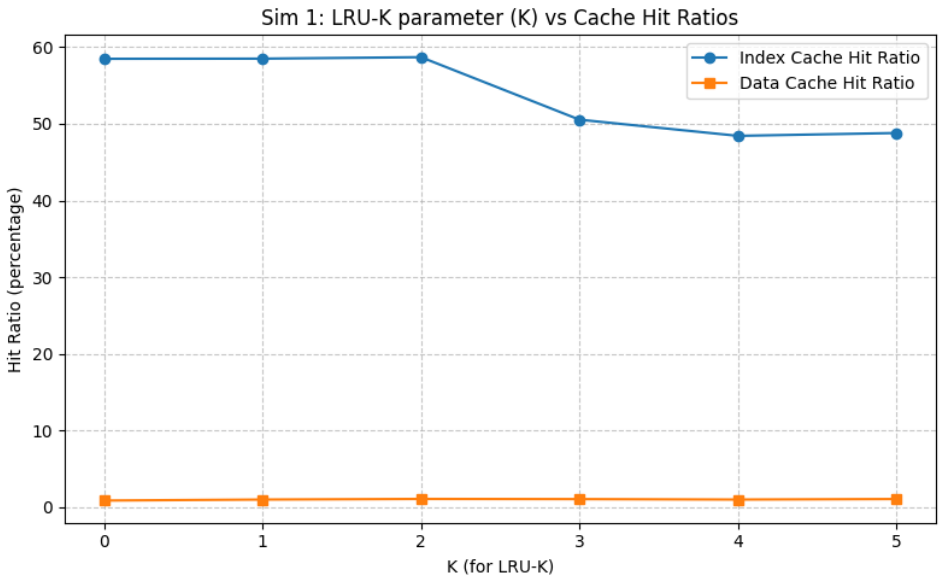

Figure 2: Simulation 1: Hit ratios v/s K

Note: $K = 0$ *corresponds to no caching (i.e., every access results in a cache miss).*

3.3 Simulation 2: Scaling Index Cache Frame Capacity

Using the optimal K value determined in Simulation 1, this test observes the marginal utility of providing more memory to the Index Cache.

Simulation Parameters:

- **Optimal** K **Used:** 2
- **Data Cache Frames:** 1,000 frames
- **Total Operations Performed:** 100000
- **Workload Distribution:** 40% Search, 30% Insert, 30% Delete

Visualizations:

1. **Graph 1: Average Latency vs. Index Frames**: A combined line graph plotting the average time taken per Insert, Delete, and Search operation against the number of Index Cache frames.

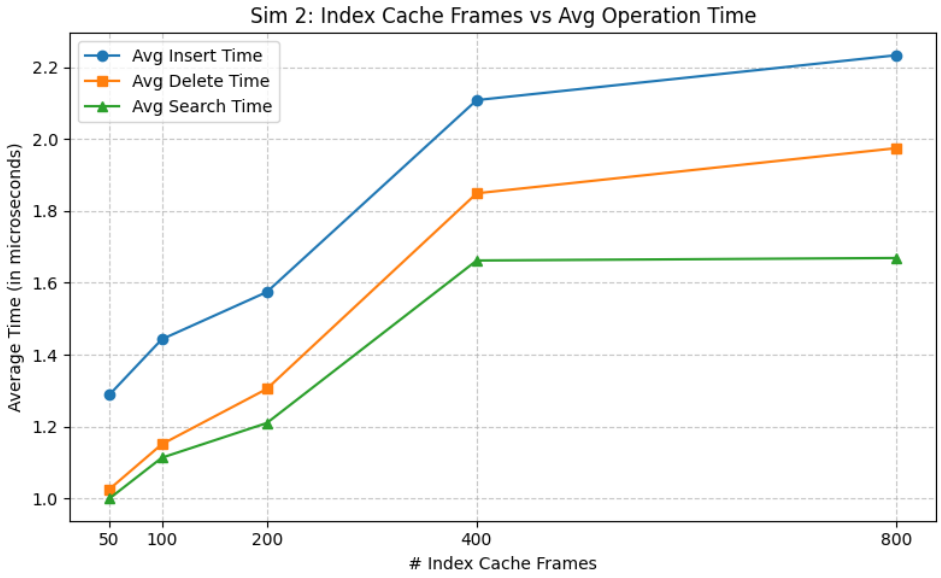

Figure 3: Simulation 2: Avg operation time v/s Index pages

2. **Graph 2: Index Hit Ratio vs. Index Frames**: A line graph plotting the total hit ratio for the Index Cache against the available frame capacity.

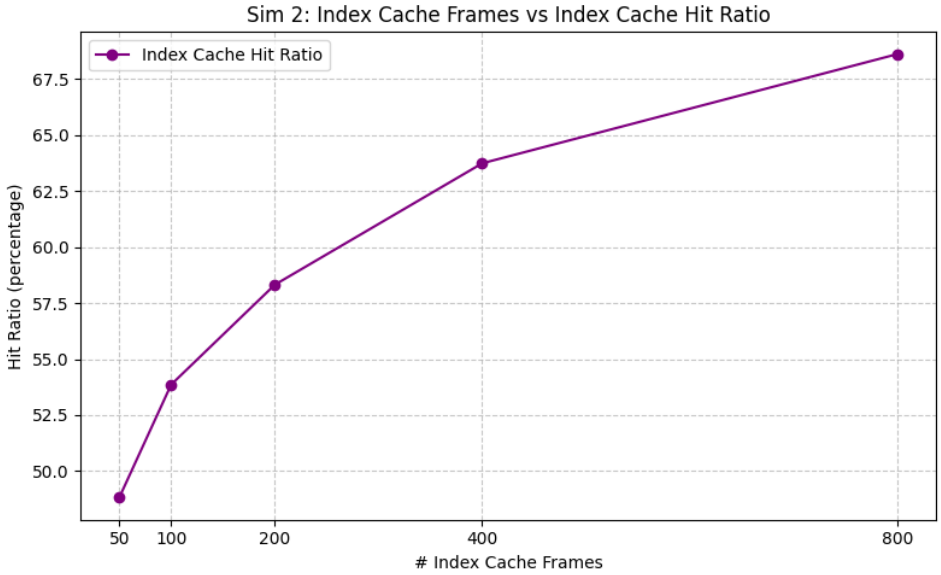

Figure 4: Simulation 2: Hit ratios v/s Index pages

3.4 Simulation 3: Concurrency Scaling via Latch Crabbing

This test measures the throughput and scalability of our Latch Crabbing implementation by varying the number of concurrent worker threads ($1, 2, 4$, and 8).

Simulation Parameters:

- **Optimal K Used:** 2
- **Index Cache Frames:** 100 frames

- **Data Cache Frames:** 1,000 frames
- **Total Operations Performed:** 100000
- **Workload Distribution:** 40% Search, 30% Insert, 30% Delete

Visualizations:

1. **Graph: Average Operation Time vs. Threads**: A plot observing how the average execution time responds as thread count increases, demonstrating the parallelism achieved through Lock Coupling.

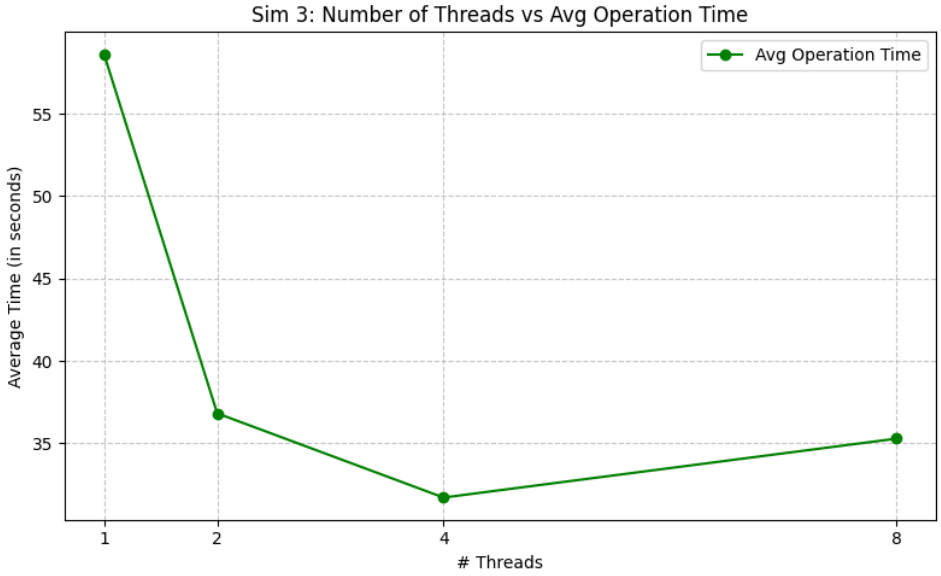

Figure 5: Simulation 3: Average Operation time(2 cores)

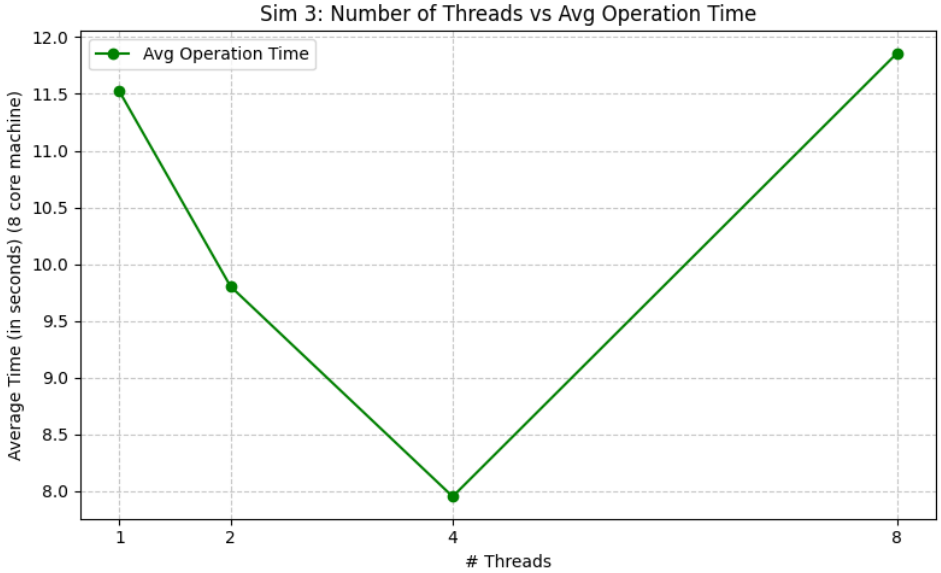

Figure 6: Simulation 3: Average Operation time(8 cores)

4 References

1. **BPTree:** https://youtu.be/K1a2Bk8NrYQ?si=GEqGgsZB2ZESZgnH
2. **Latching:** https://15721.courses.cs.cmu.edu/spring2016/papers/a16-graefe.pdf
3. **LRU-K:** https://www.geeksforgeeks.org/system-design/lru-cache-implementation/